Personalized Travel Itinerary Planner Overview Workspace Kanban Add Demo and GitHub links! Please update the demo and GitHub links so that your mentor can review and evaluate your project. Transportation individual The Personalized Travel Itinerary Planner is a full-stack, AI-powered web application that simplifies travel planning through intelligent automation and personalized recommendations. Built with a React frontend, FastAPI backend, and Supabase integration, it generates customized day-by-day itineraries based on user preferences, budget, and travel style using prompt engineering and generative AI. The system delivers optimized plans that include attractions, dining, accommodations, and transportation in a clear and structured format. With a modern SaaS-style dashboard, smart trip builder, and interactive itinerary view, the platform reduces decision fatigue and saves planning time. Users can quickly generate, save, edit, and regenerate trips while receiving curated recommendations tailored to their needs. By automating research and logistics, the application enables travelers to focus more on experiences rather than planning complexity. Scenario 1: Solo Traveler Planning a Weekend Getaway A solo traveler wants to plan a short weekend trip but has limited time to research attractions and routes. They enter their destination, budget, travel dates, and interests such as food and sightseeing into the planner. Within seconds, the system generates a balanced itinerary that optimizes travel time, suggests nearby attractions, and recommends affordable dining spots. The traveler follows a clear, structured plan and avoids spending hours searching across multiple websites, resulting in a smooth and stress-free getaway. Scenario 2: Couple Planning a Personalized Vacation A couple planning a romantic vacation wants a customized experience rather than generic tourist recommendations. They select preferences like relaxed pacing, scenic spots, and premium stays. The platform uses AI to create a day-wise itinerary including curated restaurants, unique experiences, and optimized travel routes. They can modify activities, regenerate specific days, and track estimated costs, allowing them to collaboratively refine their plan and enjoy a tailored trip that aligns with their expectations. Scenario 3: Small Group Coordinating a Multi-Day Trip A group of friends organizing a multi-day holiday faces challenges coordinating schedules, budgets, and shared interests. Using the planner’s dashboard, they create a shared trip, input group size, and combine preferences such as adventure, culture, and budget limits. The system automatically structures activities, allocates time efficiently, and provides logistics suggestions to minimize conflicts and travel delays. This centralized planning approach reduces coordination overhead and ensures everyone enjoys a well-organized and memorable group travel experience. Skills Required Python (Programming Language) Flask (Web Framework) FastAPI Generative Artificial Intelligence Artificial Intelligence Mentor No mentor assigned yet Project Stats Epics Total Epics: 10 Stories & Subtasks Total Tasks: 24 Total Subtasks: 0 Technical Architecture Technical Architecture System Requirements Instructions CPU: Minimum Intel Core i5 or AMD Ryzen 5 processor with at least 2 GHz clock speed; supports running Transformer models on CPU. RAM: Minimum 8 GB recommended since three Transformer models load into memory simultaneously at startup. Storage: At least 2 GB free disk space required for .keras model files and .pkl vocabulary files. GPU (Optional): NVIDIA GPU with CUDA support for faster inference if TensorFlow detects GPU acceleration. Network: Internet connection required for loading Google Fonts in the UI and optional Llama-3 API calls in production mode.

Node.js Installation: Install Node.js 18+ to enable backend and npm package management.https://nodejs.org/en/download FastAPI Framework:

High-performance Python framework for building RESTful APIs with async support, automatic documentation, and data validation —https://fastapi.tiangolo.com/ React.js Frontend: Build interactive UI screens using the React JavaScript framework.https://react.dev/ Supabase Setup: Create a free Supabase account for cloud database and authentication services.https://supabase.com/ GitHub Version Control: Use GitHub to manage source code, repositories, and collaboration.https://github.com/ Vercel Deployment: Deploy the frontend application easily with Vercel hosting.https://vercel.com/ Render Deployment: Host and run backend services on Render for production.https://render.com/

Description Project workflow is the step-by-step process that guides how a project is planned, developed, and completed. It helps ensure that each task is done in order and the project stays organized from start to finish.

Description 1. Cursor IDE To install Cursor, first visit its official site and download the version appropriate for your operating system (Windows, macOS, or Linux). On Windows, run the downloaded .exe installer and follow the setup wizard prompts. On macOS, open the downloaded .dmg file and drag Cursor to the Applications folder. If the system shows a security warning, allow the installation to proceed. On Linux, download the .AppImage (or .deb) version. For .AppImage, make the file executable (chmod +x ...), then run it. If needed, install libfuse2 to avoid FUSE-related errors. Once installed and launched, you can begin using Cursor's AI-powered features such as context-aware autocompletion and natural-language code generation to start writing code efficiently. 2. Github Repo To begin using GitHub, you first navigate to the official GitHub website and sign in with your credentials, or create a new account if you are a first-time user. After logging in, you can create a new repository by selecting the "New repository" option from the "+" menu located at the top-right of the interface. In the repository creation form, you provide a name, optionally add a description, choose visibility(public or private), and if desired, initialize the repository with a README or .gitignore file. Once created, GitHub displays the repository page along with a unique URL (HTTPS or SSH), which will later be used to link and push code from your local project environment. Resources No resources available Activity 1.2: Enable AI coding tools (Cursor AI agent, ChatGPT prompts). Story Duration: 1h 0m To Do PI Assigned to Piyush Pal Description In this step, AI-powered coding assistance is enabled inside the development environment to improve productivity and streamline the software development workflow. By activating the Cursor AI agent within the Cursor IDE, developers can generate code from natural language instructions, refactor existing scripts, fix errors intelligently, and receive context-aware coding suggestions. Additionally, ChatGPT prompts are integrated to support tasks such as code explanations, documentation generation, debugging guidance, and rapid prototype development. Together, these AI tools reduce manual effort and allow developers to focus more on logic and functionality while accelerating overall development involving Generative AI-based projects. Resources No resources available

ctivity 1.3: Generate folder structure using AI. Story Duration: 1h 0m To Do PI Assigned to Piyush Pal Description In this step, the initial folder structure of the project is automatically generated using AI capabilities within the Cursor IDE. Instead of manually creating each directory and configuration file, the

AI tool is prompted with the project requirements, allowing it to generate a clean, well-organized structure that follows best practices for modern development. This includes creating essential folders for source code, assets, configuration files, and environment settings, ensuring that the project remains scalable and easy to maintain. By leveraging AI for this setup, developers significantly reduce setup time, minimize structural errors, and ensure consistency from the very beginning of the project. Resources No resources available

Activity 2.1: Create Supabase project. StoryDuration: 1h 0m To Do PIAssigned toPiyush Pal Description Step 1: Create Supabase Account & Project Go to Supabase Visit: https://supabase.com/ Click "Start your project" or "Sign up" Sign Up / Sign In Sign up with GitHub, Google, or email Verify your email if needed Create New Project Click "New Project" button Fill in the details: Name: ai-symptom-checker (or any name you prefer) Database Password: Create a strong password (SAVE THIS - you'll need it) Region: Choose closest to you (e.g., Southeast Asia (Singapore)) Pricing Plan: Select Free plan Click "Create new project" Wait 2-3 minutes for the project to be created Step 2: Check Project Status: Click on “Project Status” button Verify whether the status looks Healthy or Not If not Healthy, Go to “Project Settings” which is at Left side of Dashboard -Click on “Restart Project” Wait for few minutes, Your project Status looks Healthy. Project gets Created. Resources No resources available

Activity 2.2: Create tables Story Duration: 1h 0m To Do PI Assigned to Piyush Pal Description Open SQL Editor In your Supabase dashboard, click SQL Editor in the left sidebar Click "New" button Copy the Schema Open the file: server/src/config/schema.sql Copy ALL the SQL code (from -- Supabase SQL Schema to the end) Paste and Run Paste the SQL code into the SQL Editor Click "Run" button (or press Ctrl+Enter) Wait for it to complete You should see: "Success. No rows returned" Verify Tables Created Click Table Editor in the left sidebar You should see 6 tables: submissions symptom_rules users Resources No resources available

Activity 2.3: Store API keys in .env. Story Duration: 1h 0m To Do PI Assigned to Piyush Pal Description Go to Project Root Navigate to: C:\INTERNSHIP\PROJECTS\Personalized Travel Itinerary Planner\ (The folder that contains both frontend and backend folders) Create .env File Create a new file named .env (not .env.txt) Follow the values(Refer Below Image): To take SUPABASE_URL, Go to “Project Settings”, Click on “Data API” Copy the Project URL and paste in .env file To take SUPABASE_SERVICE_ROLE_KEY and SUPABASE_ANON_KEY Go to “Project Settings”, Click on “API Keys” Check for “Legacy anon, service_role API keys” Copy anon public, service_role secret keys and paste in .env file. Resources No resources available

Personalized Travel Itinerary Planner Overview Workspace Kanban Activity 3.1: User Preference Capture Story Duration: 1h 0m To Do PI Assigned to Piyush Pal Description This activity focuses on collecting

detailed travel inputs from users through structured forms and interactive UI components. Information such as destination, travel dates, budget, number of travelers, interests, dietary needs, and travel pace is captured to build a clear understanding of user expectations. These inputs are validated and formatted into standardized request schemas to ensure consistent processing across the system. On the implementation side, the frontend provides guided forms and step-based trip builders, while the backend stores and validates the data before passing it to the recommendation engine. This ensures that every itinerary generated later is grounded in accurate, personalized information rather than generic assumptions, forming the foundation of the planning workflow. Collects user travel preferences such as destination, dates, budget, and travel style through a multi-step form. Manages step navigation and form state using React hooks like useState and useEffect. Sends trip data to backend APIs via tripsAPI to create trips and trigger itinerary generation using FastAPI services. Enhances UI/UX with icons and animations using Framer Motion and Lucide for a modern SaaS experience. Defines request and response data models using Pydantic to validate API inputs and outputs. Handles authentication schemas such as user registration, login, and JWT token responses for secure access control. Enforces field rules like email format, password length, and required attributes to ensure clean and reliable data. Acts as the data contract between frontend and backend in FastAPI, ensuring consistent request/response structures across all APIs.
Activity 3.2: Prompt Structuring Logic Story Duration: 1h 0m To Do PI Assigned to Piyush Pal Description This activity converts raw user inputs into structured prompts that can be effectively interpreted by generative AI models. Prompt engineering techniques are used to define roles, instructions, constraints, and context, enabling the AI to act like a travel expert rather than producing random suggestions. Clear formatting improves accuracy, relevance, and consistency of generated results. Technically, templates are created that dynamically inject user preferences into predefined prompt patterns. This layer acts as a bridge between the application logic and AI services, ensuring that recommendations and itineraries align closely with user needs while maintaining repeatable and optimized output. Constructs a dynamic AI prompt by combining user inputs like destination, dates, travel style, interests, budget, and group size into a structured itinerary request. Applies personalization logic by adjusting activity pacing (relaxed, moderate, fast) and incorporating dietary restrictions and mobility concerns. Formats all preferences into clear natural-language instructions to guide the AI model toward generating accurate, context-aware travel plans. Serves as the prompt-engineering layer that connects backend logic in FastAPI with the itinerary generation handled by AI services. Resources No resources available

Activity 3.3: Destination Recommendation Engine Story Duration: 1h 0m To Do PI Assigned to Piyush Pal Description This activity generates intelligent destination and activity suggestions based on the structured preferences. Instead of providing generic tourist lists, the system filters experiences according to interests, budget, and travel style to produce curated, meaningful recommendations. Dining options, attractions, and experiences are selected to match the traveler’s profile. From a system perspective, the backend processes prompts through the AI service and returns categorized recommendations to the frontend. These results are displayed as cards or lists, enabling users to review options and proceed with itinerary creation, making planning faster and more personalized. Defines all trip-related REST endpoints

using FastAPI APIRouter, handling routes like create trip, fetch trips, and generate itineraries. Connects frontend requests to backend business logic by calling trip, itinerary, history, and AI services for processing travel data. Uses request/response schemas to validate inputs and return structured outputs, ensuring consistent API communication. Orchestrates AI itinerary generation by integrating prompt builders and the AI engine, acting as the main controller between user requests and intelligent planning logic. Resources No resources available

Activity 4.1: Daily Itinerary Builder Story Duration: 1h 0m To Do PI Assigned to Piyush Pal Description This activity transforms recommended destinations into organized day-by-day schedules. Each day is structured with logical time slots such as morning, afternoon, and evening to create a clear flow of activities. The aim is to design balanced plans that maximize coverage without overwhelming the traveler. Integrates external AI/LLM providers and manages text generation for itinerary creation through a provider-based architecture. Defines an abstract base class (LLMProvider) to standardize how different models generate responses, making the system extensible and modular. Implements the Groq API client to send prompts and receive AI-generated travel plans asynchronously. Acts as the AI orchestration layer inside FastAPI, connecting prompt templates with model outputs to produce dynamic itineraries. Implementation involves generating structured itinerary objects that include activities, timings, and notes, which are then rendered in the dashboard or timeline view. This converts raw suggestions into actionable plans that users can easily follow and modify. Resources No resources available

erary Planner

OverviewWorkspaceKanban

Activity 4.2: Route & Time Optimization

StoryDuration: 1h 0m

To Do

PIAssigned to**Piyush Pal**

Description

This activity improves efficiency by minimizing travel time between locations and arranging activities in geographically sensible sequences. The system considers distance, pacing, and time constraints to reduce unnecessary backtracking and fatigue, creating smoother daily schedules.

Builds structured AI prompts by converting user inputs (destination, dates, interests, pace, budget, constraints) into clear natural-language instructions.

Applies prompt engineering rules to control itinerary style, pacing, and personalization for more accurate and relevant AI outputs.

Standardizes the format of requests sent to the AI layer, ensuring consistent and repeatable results across generations.

Acts as the prompt construction layer that connects business logic in **FastAPI** with the LLM provider to generate smart travel itineraries.

On the backend, optimization logic adjusts ordering and timing before finalizing the itinerary. The frontend then displays these optimized routes visually, helping users experience a practical and realistic travel plan rather than a random sequence of stops.

Resources

No resources available

Activity 4.3: Vibe-Based Personalization

StoryDuration: 1h 0m

To Do

PIAssigned to**Piyush Pal**

Description

This activity enhances the emotional and experiential aspect of planning by adapting itineraries to different travel styles such as relaxed, adventure, luxury, or budget. Users can request changes, regenerate plans, or refine specific days through conversational interactions, making the system flexible and adaptive.

Implements a multi-step trip creation form to collect destination, dates, group size, budget, and preferences from users.

Uses **React** hooks (useState, useEffect) for managing form state, navigation, and loading behavior.

Calls backend APIs through tripsAPI to create trips and trigger itinerary generation from the server.

Enhances user experience with animations using **Framer Motion** and icons from **Lucide** for a modern SaaS-style interface.

Technically, prompts are dynamically adjusted based on selected vibes and feedback, and new itineraries are regenerated while preserving previous context. This iterative refinement ensures the plan evolves with user preferences rather than remaining static.

Activity 5.1: Trip Save & Management

StoryDuration: 1h 0m

To Do

PIAssigned to**Piyush Pal**

Description

This activity introduces persistent storage, allowing users to save, retrieve, edit, and delete their trips and itineraries. Instead of temporary results, all travel plans are securely stored in the database, enabling long-term access and management through a dashboard.

Initializes and manages the database connection using Supabase by creating both public and service-role clients.

Provides reusable methods (get_client, get_service_client) to securely access the database across services and routes.

Separates normal user operations from admin-level actions through the service role key for elevated permissions.

Acts as the centralized database layer integrated with FastAPI, ensuring consistent and scalable data access throughout the backend.

The backend handles CRUD operations, while the frontend provides trip history pages and management controls. This transforms the application from a simple generator into a usable, real-world planning tool with continuity and organization.

Activity 5.2: Share & Export Module

StoryDuration: 30m

To Do

PIAssigned to**Piyush Pal**

Description

This activity enables users to share their travel plans with friends or collaborators and export them in readable formats. Shareable summaries, downloadable files, or links make itineraries practical for real-life usage and group coordination.

Displays detailed trip and itinerary information by fetching data using the trip ID from the route parameters.

Uses **React** hooks (useState, useEffect, useParams) to manage trip data, loading states, and navigation.

Calls backend services through tripsAPI to retrieve itineraries and supports actions like download, edit, or regenerate plans.

Improves the travel dashboard experience with layout components, animations via **Framer Motion**, and icons from **Lucide**.

From a technical standpoint, formatted itinerary data is converted into compact summaries or structured outputs that can be displayed or exported. This improves collaboration and enhances the overall usability of the platform beyond individual planning.

Activity 5.3: Testing & Production Deployment

StoryDuration: 1h 0m

To Do

PIAssigned to**Piyush Pal**

Description

This activity ensures the reliability and scalability of the application through systematic testing and optimization. Different traveler scenarios are validated to confirm accuracy, prompt effectiveness, and system stability, while performance issues are identified and resolved.

Displays detailed trip and itinerary information by fetching data using the trip ID from the route parameters.

Uses **React** hooks (useState, useEffect, useParams) to manage trip data, loading states, and navigation.

Calls backend services through tripsAPI to retrieve itineraries and supports actions like download, edit, or regenerate plans.

Improves the travel dashboard experience with layout components, animations via **Framer Motion**, and icons from **Lucide**.

Finally, the project is prepared for production with environment configurations, build optimization, and cloud deployment. This guarantees that the application runs smoothly in real-world conditions and is ready for continuous use and future enhancements.

Activity 5.4: Environment & Configuration Management

StoryDuration: 30m

To Do

PIAssigned to**Piyush Pal**

Description

This activity focuses on securely managing application configuration and external service credentials required for the system to run across development, staging, and production environments. It includes storing API keys, database URLs, Supabase credentials, and AI provider tokens inside environment variables instead of hardcoding them in the source code, ensuring security, portability, and deployment readiness. Proper configuration handling enables smooth integration between backend services, frontend APIs, database access, and AI itinerary generation.

Generate GROQ API

Click on the link (https://in.search.yahoo.com/search?fr=mcafee&type=E210IN714G0&p=groq+api+key)

Then click on “API Keys-Groq Cloud”.

Click on “Get API key” from the right navigation menu.

Now click on “Create API key”. (Refer the above image)

Copy the API key.

SECRET_KEY (JWT / Authentication Key)

Used for signing tokens and securing authentication in **FastAPI**.

Steps

Open terminal inside backend folder

Run:

python -c "import secrets; print(secrets.token_urlsafe(32))"

Copy the generated string

Paste into .env:

Hugging Face Access Key (Free LLM API)

Used when selecting **Hugging Face** models instead of Groq.

Steps

Visit:
https://huggingface.co/settings/tokens

Login → **New Token**

Select:

Role → **Read**

Name → travel-planner

Copy token

Add to .env

Ollama (Local LLM – Offline Mode)

Used for running models locally without internet using **Ollama**.

Steps

1. Install Ollama

Download:
https://ollama.com/download

Install and restart terminal.

2. Pull a model

Example:

ollama pull llama3

or

ollama pull mistral

3. Start server (auto-starts usually)

Test:

ollama run llama3

4. Configure .env

→ Place all the API Keys into .env

backend > .env

```
SUPABASE_URL=https://kdmwokydfvxppyzccjks.supabase.co
SUPABASE_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsIn
SUPABASE_SERVICE_ROLE_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzd
# ============ JWT AUTHENTICATION ============
# Generate a strong secret key (minimum 32 characters, use random string)
# You can use: python -c "import secrets; print(secrets.token_urlsafe(32))"
SECRET_KEY=d3kL9mP2vQ8xR4sT7uY1wZ5aB6cD9eF2gH3iJ4kL5mN6oP7qR8sT9uV0wX1yZ2
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=30

# ============ AI MODEL CONFIGURATION ============
# Choose ONE: groq, huggingface, or ollama
# Groq is RECOMMENDED - Free tier with generous limits

# For Groq (RECOMMENDED)
AI_PROVIDER=groq
GROQ_API_KEY=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
HUGGINGFACE_API_KEY=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
OLLAMA_BASE_URL=http://localhost:11434



# For Ollama (Local - No API key needed)
# AI_PROVIDER=ollama
# OLLAMA_BASE_URL=http://localhost:11434

# ============ SERVER CONFIGURATION ============
DEBUG=True
ENVIRONMENT=development
BACKEND_URL=http://localhost:8000
FRONTEND_URL=http://localhost:5173

# ============ REDIS CONFIGURATION (Optional) ============
# For caching (optional, leave empty if not using)
REDIS_URL=

# ============ LOGGING ============
LOG_LEVEL=INFO
```

Activity 6.1: Deploy Backend to Render

StoryDuration: 30m

To Do

PIAssigned to**Piyush Pal**

Description

The backend server is deployed to Render, which hosts Node.js APIs with automatic scaling and continuous deployment from GitHub. The environment variables are safely configured in Render's dashboard, and a live web service URL is provided to allow the frontend to communicate with all backend endpoints over the internet.

Go to Render Cloud https://render.com/

Go to DashBoard

Click on “Add New” → “Web Service”

Import your project from Github

Configure the Service

Name: Personalised Travel (or your preferred name)

Environment: Node

Region: Choose closest to your users

Branch: main (or your default branch)

Root Directory: Leave empty (or backend if deploying from backend folder)

Build Command: pip install -r requirements.txt

Start Command: gunicorn app.main:app -k uvicorn.workers.UvicornWorker --bind 0.0.0.0:$PORT

In Render dashboard, go to Environment variables tab and add from .env :

NODE_ENV=production SUPABASE_URL=your_supabase_project_url SUPABASE_SERVICE_ROLE_KEY=your_supabase_service_role_key JWT_SECRET=generate_a_strong_random_string_here PORT=5000 and etc...needed

- Click on Add Variables

Click on Deploy Web Service

Wait for the Deployment service to be “Live”

Activity 6.2: Deploy Frontend to Vercel

StoryDuration: 30m

To Do

PIAssigned to**Piyush Pal**

Description

In this activity, the React frontend is deployed using Vercel, a fast and developer-friendly platform optimized for modern JavaScript frameworks. The production build is generated and pushed to GitHub, and Vercel automatically builds and deploys the interface. After successful deployment, Vercel provides a secure HTTPS live URL for users to access the application from anywhere.

Go to Vercel Deploy https://vercel.com/

Click on “Add New”--> “Project”

Import your project from GitHub

Edit the Root Directory

Set to “Frontend” → Then Click on “Continue”

- Enable “Build and Output Settings” and give the commands (refer below lines)

Build Command: 'npm run build'

Output Directory: 'dist'

Install Command: 'npm install'

Set Environment Variable Key as “VITE_API_URL” and

Assign Value as “ your-render-backend-url.onrender.com/api”

-Click on “Deploy”

-Production Deployment is done

-Click on the URL which is at **Domains** section (Refer above Image)

- Copy the https URL “https://personalised-travel-itinerary-plann.vercel.app”
-Place the URL in Render→Environment→ Environment Variables→ FRONTEND_URL

Click on “Save,rebuild and Deploy” button

Wait for the render Deployment to be “Live” again

Now, the Render and Vercel is connected to Supabase and Deployment is done.

Activity 7.1: Perform Functional Testing

StoryDuration: 30m

To Do

PIAssigned to**Piyush Pal**

Description

This activity verifies that all core features of the travel planner work correctly end-to-end by testing user registration/login, preference collection, itinerary generation, trip saving, editing, and sharing workflows. Each API endpoint in the **FastAPI** backend and every UI interaction in the **React** frontend is validated to

ensure correct inputs, expected outputs, proper error handling, and smooth integration between the database, AI services, and user interface.

Activity 7.2: Validate Database Updates

StoryDuration: 30m

To Do

PIAssigned to**Piyush Pal**

Description

This activity ensures that all create, read, update, and delete operations correctly reflect in the database by verifying that user accounts, trips, and itineraries are accurately stored, retrieved, and modified without data loss or inconsistencies. It involves checking table records, foreign key relationships, and row-level security rules in **Supabase**, confirming that the **FastAPI** backend properly synchronizes application actions with persistent storage.

Outputs

StoryDuration: 1h 0m

To Do

PIAssigned to**Piyush Pal**

Description

Landing page:

Login page:

Dashboard:

Day wise Plan:

Explore:

Navigation plan on Selected Trip:

Saved Places:

Trip History:

Resources

No resources available

Description

The Personalized Travel Itinerary Planner successfully demonstrates how Generative AI can be integrated with modern full-stack technologies to deliver intelligent, user-centric travel experiences. By combining a responsive interface built with **React** and a scalable backend powered by **FastAPI**, the system collects user preferences, processes them through structured prompts, and generates customized, day-by-day itineraries using AI models. The platform replaces manual planning and generic suggestions with optimized, context-aware recommendations tailored to budget, travel style, and personal interests.

From a product perspective, the solution evolves beyond a simple planner into a SaaS-ready application with authentication, trip management, persistent storage via **Supabase**, and deployment support for

cloud environments. It ensures reliability through testing, secure configuration management, and modular service architecture, making the system extensible for future enhancements such as real-time collaboration, smarter recommendations, and additional AI providers. Overall, the project delivers a practical, scalable, and intelligent travel planning experience that bridges AI innovation with real-world usability.